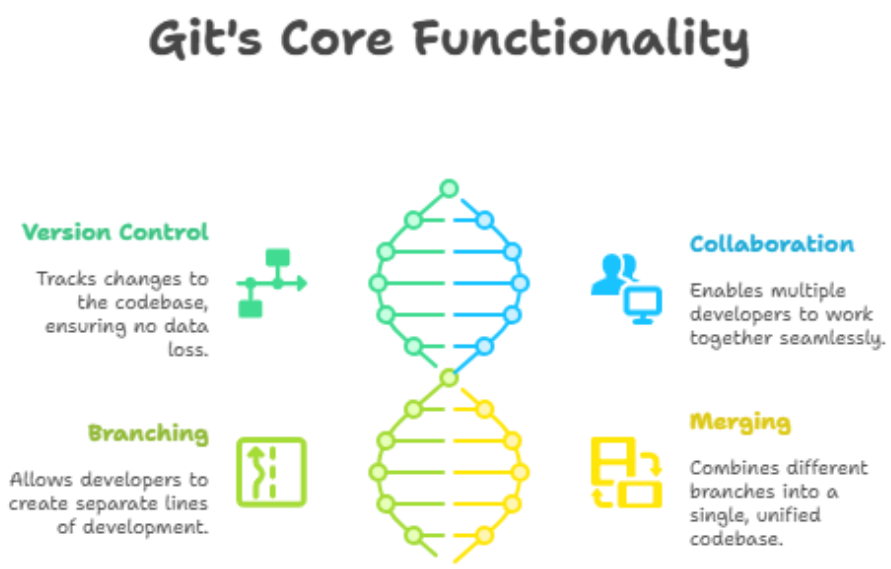
Git

What is Git?

Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency. It allows multiple developers to collaborate on a project by tracking changes to the codebase, managing different versions of files, and enabling workflows like branching and merging.



Key Features of Git:

- **Distributed System**: Every developer has a complete local copy of the repository, including the entire history of changes.
- **Branching and Merging**: Git makes it easy to create branches for separate work streams and later merge them back into the main code base.
- **Performance**: Git is fast in terms of committing changes, branching, and merging.
- **Data Integrity**: Git ensures the integrity of your data using cryptographic hashing (SHA-1) to track changes.
- **Flexibility**: It supports non-linear development processes through its branching and merging features.
- **Collaboration**: Tools like GitHub, Git Lab, and Bitbucket enhance collaboration by hosting Git repositories.

What is a Version Control System (VCS)?

A **Version Control System** is a tool for tracking changes to files over time. It helps developers manage and collaborate on a code base by maintaining a history of changes, enabling reversion to previous versions, and managing simultaneous updates by multiple contributors.

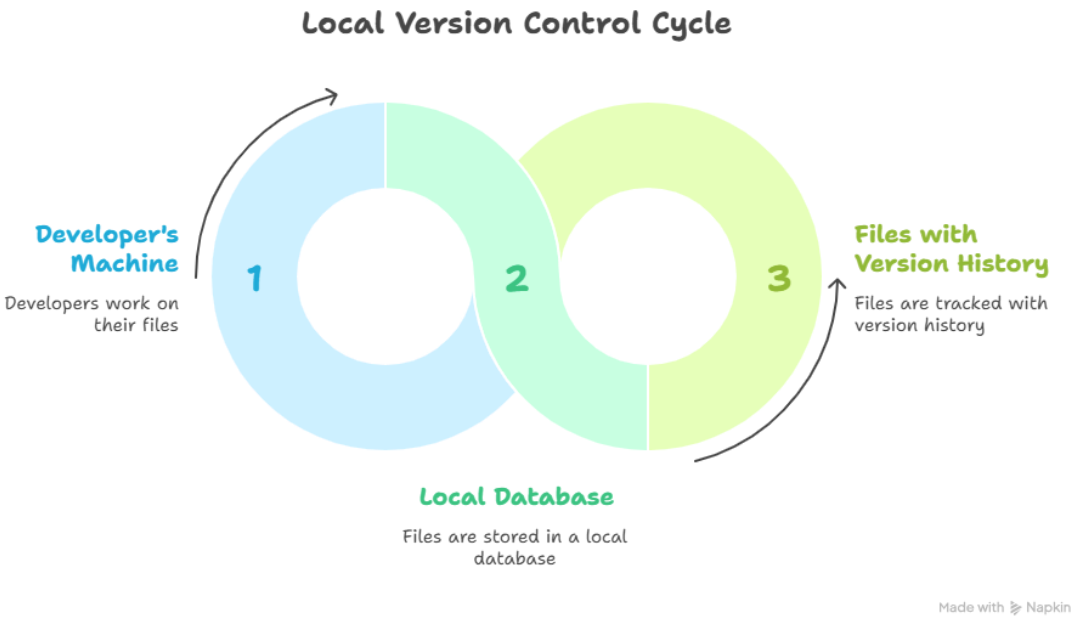
Types of Version Control Systems (VCS)

Version Control Systems are tools used to manage changes in a project over time. There are three primary types:

1. Local Version Control Systems

- **How They Work**:
  - These systems store the version history of files on a developer's local computer. Changes are tracked in a local database.
  - Developers work independently and rely solely on their local machines for version tracking.

- **Example**: **RCS (Revision Control System)**.
    - RCS stores changes as a series of patches (differences between file versions). To view an older version, the system applies these patches in reverse order.
- **Drawbacks**:
    - Collaboration is difficult because each developer works in isolation, with no centralized way to share updates.
    - Risk of data loss if the local machine fails.

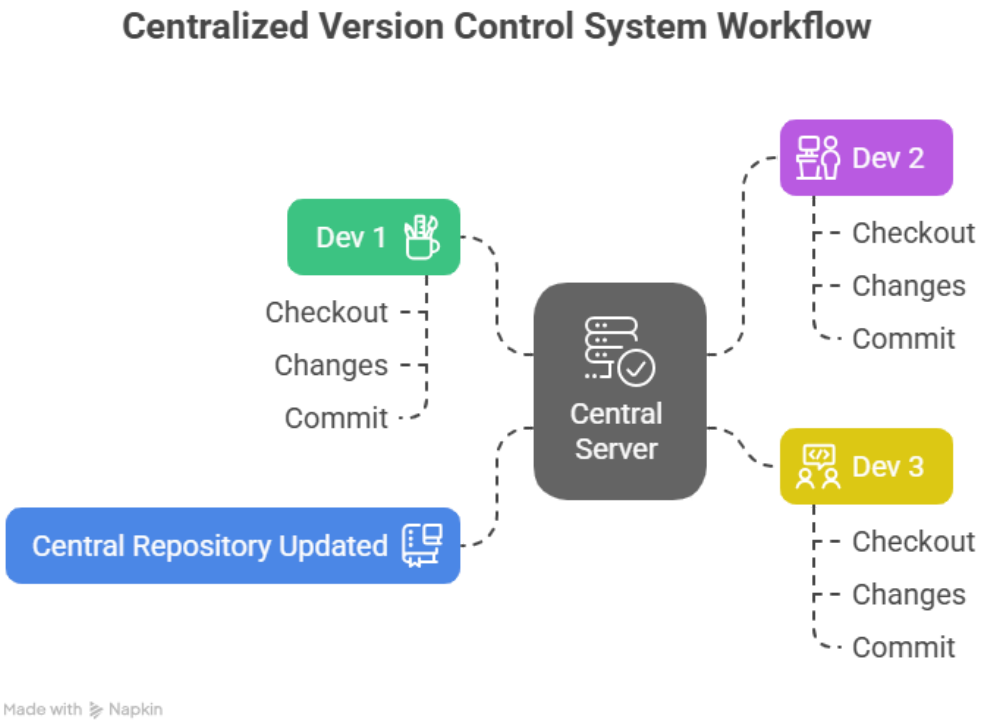


2. Centralized Version Control Systems (CVCS)

- **How They Work**:
    - All versioned files are stored on a **central server**. Developers "check out" files from the server to their local machine, make changes, and then upload (commit) the changes back to the server.
- **Examples**: **CVS (Concurrent Versions System)**, **SVN (Apache Subversion)**.
- **Benefits**:

- Facilitates collaboration by providing a single source of truth for the project.
- Easy to manage and enforce access control since everything is stored in one place.

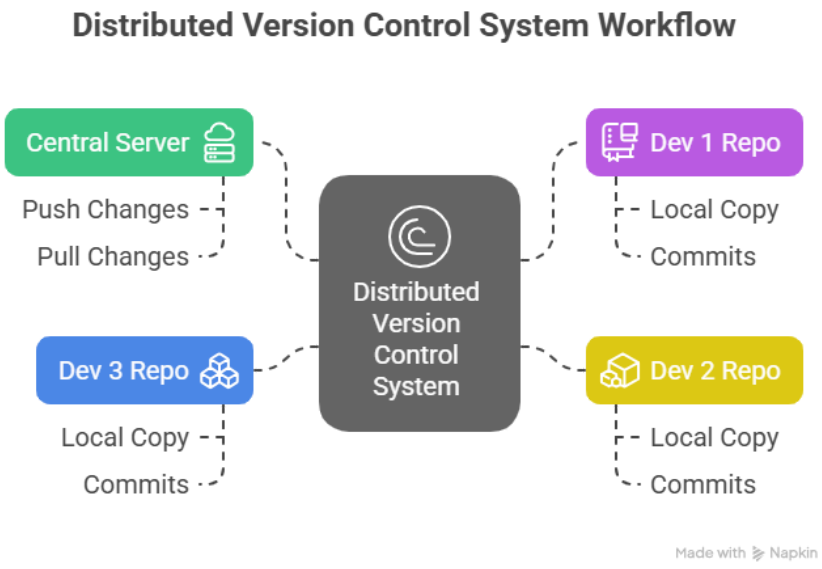
- **Limitations**:
  - **Single Point of Failure**: If the central server goes down, developers cannot collaborate, access the history, or commit changes.
  - **Dependency on Connectivity**: Developers need to be connected to the central server to access or commit changes.



3. Distributed Version Control Systems (DVCS)

- **How They Work**:
  - Every developer has a **complete copy of the repository**, including the full version history, on their local machine. Changes can be committed locally, and updates can be synchronized between repositories using "push" or "pull" operations.
- **Examples**: **Git**, **Mercurial**.

- **Benefits**:
  - **No Single Point of Failure**: Since every contributor has the full repository, work can continue even if the central server fails.
  - **Offline Work Capability**: Developers can commit changes, view history, and experiment with branches without needing an internet connection.
  - **Improved Collaboration**: Developers can share changes directly with each other, bypassing the central server if necessary.
- **Drawbacks**:
  - Requires more disk space and computational resources since each local copy contains the entire project history.



Git Repository Basics

What is a Git Repository?

A **Git repository** is a storage space where Git tracks and manages your project's files, along with their entire history of changes. It acts as the central location for your project, enabling version control and collaboration among team members.

A repository typically contains:

- **Tracked files**: Source code, configuration files, etc.
- **History**: All commits made to the repository.
- **Branches**: Separate workstreams (e.g., `main`, `feature-branch`).
- **Tags**: Marked points in history (e.g., releases).
- **Staging area**: Tracks changes to be committed.

Repositories can be created locally on your machine or hosted remotely for collaboration.

Types of Repositories:

A
bare repository and a **non-bare repository** are two types of Git repositories with distinct purposes and use cases. The difference primarily lies in how they store files and how they are used.

1. Non-Bare Repository

- **Definition**: A regular Git repository used for development. It contains:
  - A `.git` folder: Stores Git's metadata and history.
  - A working directory: Contains the actual project files you work on.
- **Purpose**:
  - Designed for day-to-day development activities like coding, committing changes, and testing.
  - You can directly modify, add, and delete files in the working directory.
- **Example**:

```
git init my-project
```

This creates a non-bare repository with both .git and the working director
y.

---

2. Bare Repository

- **Definition**: A repository without a working directory. It only contains:
  - The contents of the `.git` folder, including the version history and metadata.
  - No editable working directory with actual project files.
- **Purpose**:
  - Designed for **collaboration and sharing** (e.g., as a central remote repository).
  - Developers clone or pull from a bare repository, but they don't work directly in it.
- **Example**:

```
git init --bare my-project.git

This creates a bare repository named my-project.git.
```

Key Differences Between Bare and Non-Bare Repositories

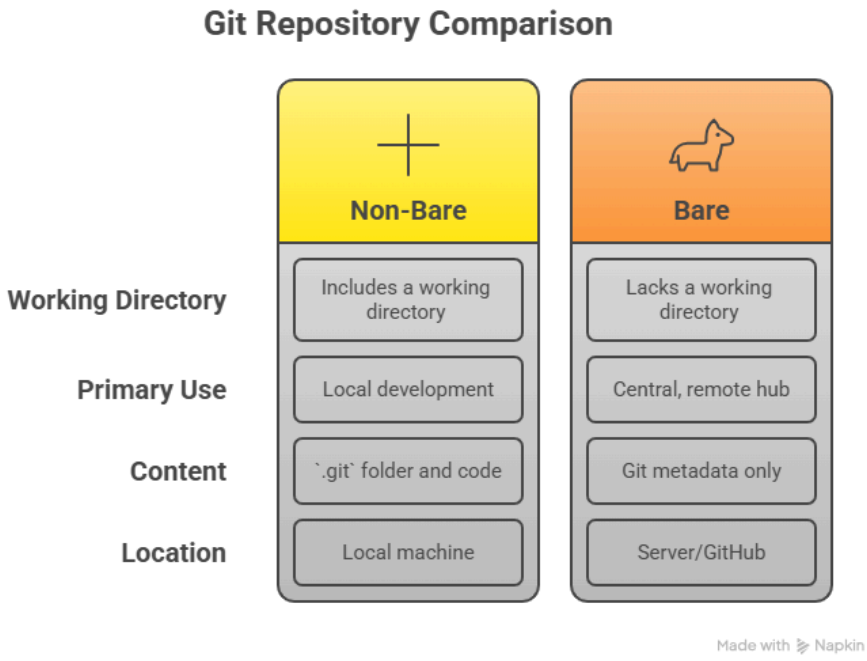
Feature	Bare Repository	Non-Bare Repository
Working Directory	No	Yes
Editable Files	Not present	Present (used for coding)
Purpose	Collaboration/remote repository	Local development
Direct Commits	Not allowed	Allowed
Typical Location	Server or remote repository	Local machine

When to Use Each

1. **Bare Repository**:
   - Use as a **central remote repository** for collaboration.
   - Ideal for hosting on platforms like GitHub, GitLab, or a private server.

- Example: `origin` in `git remote add origin <url>` typically points to a bare repository.

2. **Non-Bare Repository**:
   - Use for **local development** and working on code.
   - Every developer's local clone is a non-bare repository.



Initializing a Repository (`git init`):

The `git init` command is used to create a new Git repository. This command initializes a Git repository in your current directory, which allows you to start tracking changes to your files with Git.

Steps to Initialize a Git Repository

1. **Navigate to Your Project Directory**:
   Open a terminal and use the
   `cd` command to navigate to the directory where you want to initialize the Git repository.

```
cd /path/to/your/project
```

2. **Run the `git init` Command**:
   Execute the following command to initialize the Git repository:

   ```
   git init
   ```

   This will create a hidden `.git` directory in your project folder. This directory contains all the metadata and history of the repository.

3. **Verify Initialization**:
   To confirm that the repository has been initialized, run:

   ```
   ls -a
   ```

   You should see a `.git` directory in the output.

Configuring Git (`git config`):

To effectively use Git, you need to configure it for your environment. Configuration involves setting up your identity, preferences, and repository-specific or global settings.

Basic Git Configuration
Set Up Your Identity Git uses your name and email address to label your commits. Configure these globally or per repository.

Globally (applies to all repositories):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Per Repository (applies only to the current repository):

```
git config user.name "Your Name"
git config user.email "
```

`your.email@example.com "`

Check Configuration To view your current configuration:

```
git config --list
```

For detailed information:

```
git config --list --show-origin
```

Common Git Configurations:

1. **Default Text Editor**
   Set your preferred text editor for Git commands like `git commit`.

   ```
   git config --global core.editor "vim"
   ```

2. **Default Branch Name**
   By default, Git creates a branch named `master`. You can change it to something like `main`

   ```
   git config --global init.defaultBranch main
   ```

3. **Enable Colored Output**
   Make Git commands easier to read by enabling color coding.

   ```
   git config --global color.ui auto
   ```

4. **Set Up Aliases**
   Simplify Git commands with aliases.

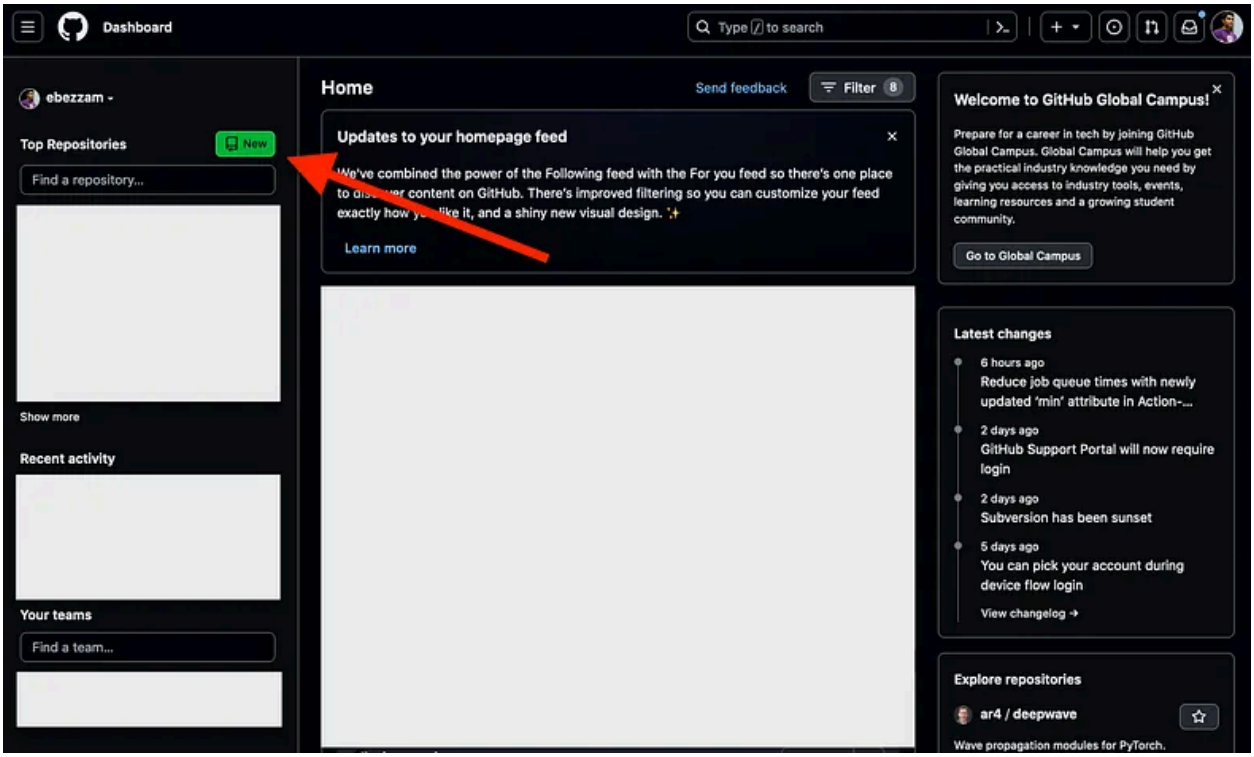
   ```
   git config --global alias.st status
   git config --global alias.co checkout
   git config --global alias.br branch
   git config --global alias.ci commit
   ```

5. **Cache HTTPS Credentials**
   Avoid entering your username and password repeatedly by caching them.

   ```
   git config --global credential.helper wincred
   ```

Creating a GitHub repository

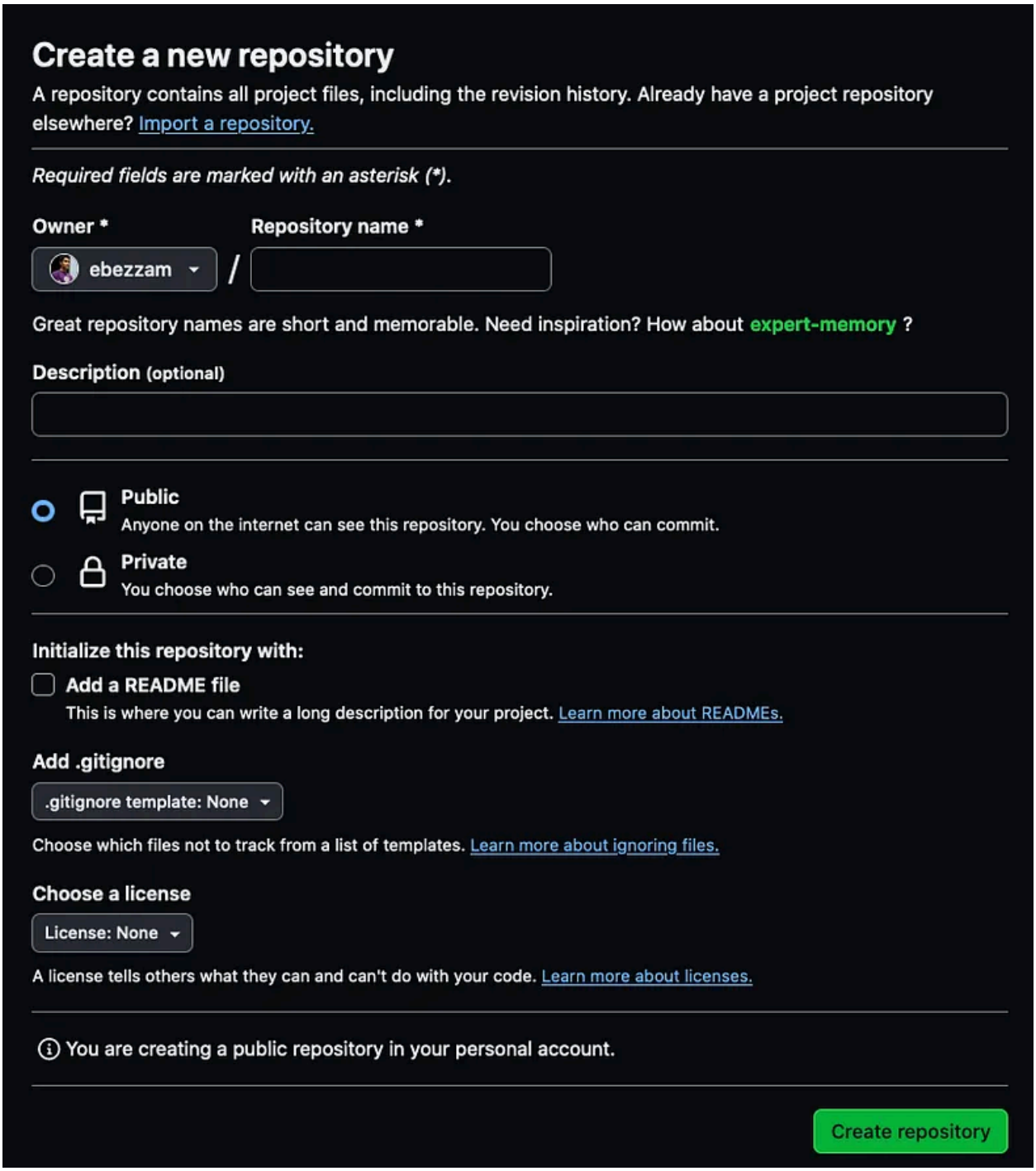
First thing of course is to log in to GitHub. If you don’t have an account, you can create one for free. Once you’re logged in, you should see a page like this:



We’ll go ahead and click on the green “New” button to create a new repo.

Initialize your repo settings

You’ll now be taken to a page that looks like this:



1) Repository name and description

This is the name of your repo. It will be used in the URL of your repo, so it's best to keep it short and simple. You can use hyphens to separate words, but not spaces. For example, if you want to create a repo for your school project, you could name it `my-school-project`

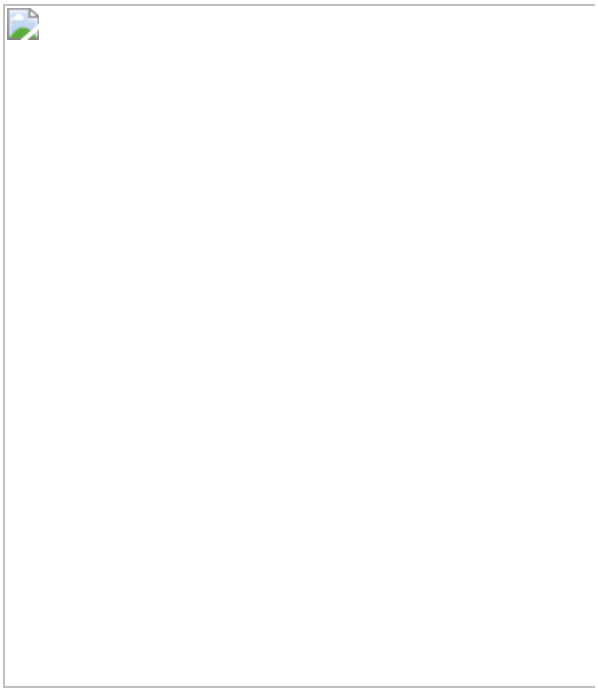
And you can optionally provide a description of your repo. This will only be displayed on the repo’s homepage on GitHub.

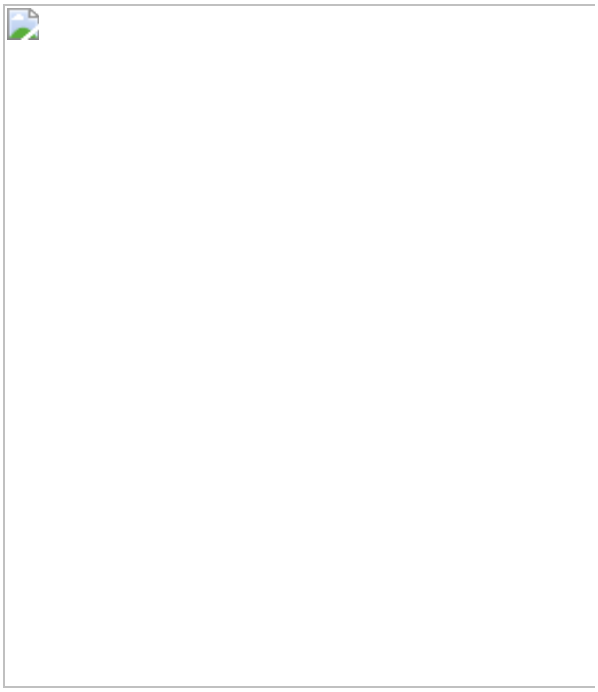
2) Public or private?

“Back in my day”, GitHub only allowed public repos for free, meaning anyone could view the contents of your repo. This was great for open-source projects, but not so great if you wanted to privately work on your code till it was clean and useable enough. GitHub now allows you to have **unlimited** private repos.

I recommend keeping your repo private until you’re ready to share it with the world. You can always change this setting later.

3) Initialize with “boilerplate” files





A **README** contains important information about your project (installation, how to use, etc). The file name is typically in all-caps to grab the user’s ATTENTION, as the first thing to look at. On GitHub, the README serves as the homepage for your repo. It will be displayed on the repo’s homepage, and will be the first thing people see when they visit your repo. A common syntax for README files is Markdown, and if you decide to, GitHub will add a `README.md` file (namely a Markdown file). It’s a good idea to have a README, so I recommend checking this box. You can always delete/change it later, e.g. if you make documentation with Sphinx and will need a `README.rst` file instead of a `README.md` file as created by GitHub (confused? more on that in another post!).

A **.gitignore** file is also really useful to have! As the name says, it tells the `git` protocol to ignore certain files. The templates that GitHub provides are pretty good, so I recommend picking the one for the primary programming language of your project. For example, for Python, it will ignore files that end in `.pyc` (compiled Python files) and `.env` (environment variables). You can always add/remove files from the `.gitignore` file later.

And that's it! Go ahead and press the green "Create repository" button. You'll be taken to your repo's homepage, which will look something like below.

Remote Repositories

1 . **What are Remote Repositories**?

A **remote repository** is a version of your project hosted on a server, often used for collaboration. It allows multiple people to work on the same project by synchronizing changes between their local repositories and the remote repository.

- **Popular Remote Hosting Services**: GitHub, GitLab, Bitbucket, Azure DevOps.
- **Remote URLs**: Remotes are identified by their URLs, which can use protocols like HTTPS or SSH.
- **Common Remote Names**:
  - `origin`: The default name given when cloning a repository.
  - `upstream`: Often used to track the original repository in forked projects.

A. `origin`

- **What it is**:

  When you clone a repository, Git automatically assigns the name `origin` to the remote URL from which you cloned the repository. It serves as the default remote name.

- **Purpose**:

  It is used to reference the original repository you cloned from, typically for pulling and pushing changes.

- **Example**:

  After cloning:

  ```
  git clone https://github.com/username/repo.git
  cd repo
  git remote -v
  ```

  Output:

  ```
  origin  https://github.com/username/repo.git (fetch)
  origin  https://github.com/username/repo.git (push)
  ```

  Usage:

- Pull & Push changes from the original repository:

```
git pull origin main
git push origin main
```

B. Upstream:

The term upstream is commonly used when you fork a repository. Your forked repository will have its remote URL named origin, but the original repository (from which you forked) is referred to as upstream.

Purpose:
upstream helps you track and sync changes from the original repository (the source of your fork). It is manually added as a remote after you fork a repository.

Setting Up upstream: After cloning your fork:
git remote add upstream
https://github.com/original-user/repo.git

Usage:

Fetch updates from the original repository:

git fetch upstream

Merge changes from the upstream branch into your local branch:
git merge upstream/main

Sync your fork with the original repository:
git pull upstream main

2. Fetching Changes (`git fetch`)

The `git fetch` command retrieves changes from a remote repository without altering your working directory. This allows you to review changes before merging them into your local repository.

`git fetch <remote-name>`

git fetch origin

git fetch --all (To fetch updates from all remotes)

git fetch <remote-name> <branch-name> (To fetch Specific branch)

3.Pulling Changes (`git pull`)

The `git pull` command fetches changes from a remote repository and merges them into your local branch. It combines `git fetch` and `git merge`.

Basic Usage
git pull <remote-name> <branch-name>

Ex :

git pull origin main

Rebase While Pulling

To rebase instead of merging:

`git pull --rebase <remote-name> <branch-name>`

4.Pushing Changes (`git push`)

To push all local changes to remote repository branch.

To push all local branches

`git push --all <remote-name>`

Forcibly push changes (use cautiously):

`git push --force`

5.Tracking Remote Branches

A **tracking branch** is a local branch that tracks changes from a remote branch. It helps synchronize changes between your local and remote repositories.

Setting Up a Tracking Branch

When cloning a repository, the default branch is automatically tracked. To manually set up a tracking branch:

```
git branch --set-upstream-to=<remote-name>/<branch-name>
```

Example:

```
git branch --set-upstream-to=origin/main
```

Check Tracking Status

To see which remote branch your local branch is tracking:

```
git status
```

List All Tracking Branches

```
git branch -vv
```

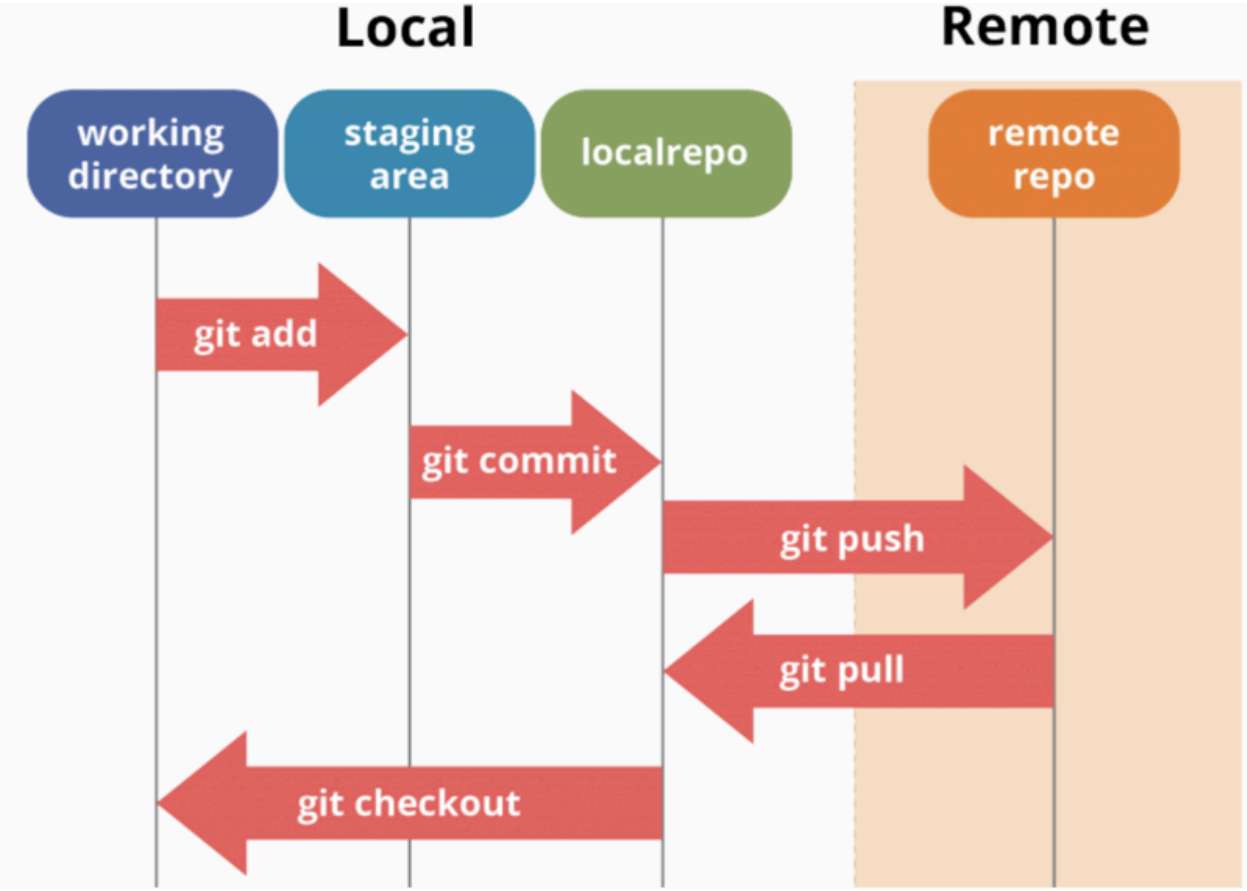
Fetch Tracking Information

Fetch updates for all tracking branches

:

```
git fetch
```

Understanding the Git Workflow:



This diagram illustrates the basic Git workflow between local and remote repositories:

Local Environment (left side):

- Working Directory: Where you edit and create files
- Staging Area: Where changes are prepared for committing
- Local Repository: Where committed changes are stored

Remote Environment (right side):

Remote Repository: The shared repository (like on GitHub)

The workflow follows these steps:

git add: Moves changes from working directory to staging area

git commit: Saves staged changes to local repository

git push: Uploads local commits to remote repository

git pull: Downloads changes from remote repository to local

git checkout: Switches between different versions or branches

Basic Git Workflow

1. **Clone the Repository**: Start by copying an existing repository (local or remote).

```
git clone <repository-url>
```

2. **Create a Branch**: Work on a new feature or bug fix without affecting the main branch.

```
it checkout -b <branch-name>
```

3. **Work on Changes**: Modify the files as needed.
4. **Stage Changes**: Add the modified files to the staging area.

```
git add <file-name> # Add specific files
git add .           # Add all changes
```

5. **Commit Changes**: Save the changes to the branch with a descriptive message.

```
 commit -m "Your commit message here"
```

6. **Pull Latest Changes (if working with a remote repository)**: Ensure your branch is up-to-date with the remote branch.

```
git pull origin <branch-name>
```

7. **Push Changes**: Upload your branch to the remote repository.

```
git push origin <branch-name>
```

8. **Create a Pull Request (PR)**: Request to merge your branch into the main branch via a remote platform like GitHub, GitLab, or Bitbucket.
9. **Review and Merge**: Team members review the PR and merge it into the main branch once approved.
10. **Delete the Feature Branch (Optional)**: After merging, clean up by deleting the branch.

```
git branch -d <branch-name> # Local branch
git push origin --delete <branch-name> # Remote branch
```

Viewing Commit History(`git log` , `git show`)

git log

The
`git log` command shows the commit history of the current branch.
`git log`

- This shows the commits in reverse chronological order, starting with the most recent commit at the top.
- Each commit will include information like:
  - Commit hash (e.g., `a1b2c3d`)
  - Author
  - Date

- Commit message

commit a1b2c3d4e5f6g7h8i9j0k (HEAD -> main)
Author: Your Name
your.email@example.com
Date: Tue Jan 6 14:15:20 2025 +0000

Fixed bug in user authentication

commit z9y8x7w6v5u4t3s2r1q0p232435467
Author: Your Name
your.email@example.com
Date: Mon Jan 5 11:10:30 2025 +0000
Initial commit

Common `git log` Options:

- `git log --oneline`: Shows each commit in a simplified, one-line format with just the commit hash and message.
- `git log --graph`: Displays a graph of commits and their relationships (useful for visualizing branching).
- `git log --author=<name>`: Filters commits by a specific author.
- `git log --since=<date>`: Shows commits since a specific date.
- `git log --until=<date>`: Shows commits until a specific date.
- `-oneline`: Show each commit as a single line.

  ```
  git log --oneline
  ```

  Example:

  ```
  abc1234 Fix typo in README
  ```

```
def5678 Add login feature
```

- `-graph`: Display the commit history as a graph of branches.

```
git log --oneline --graph
```

- `-author=<name>`: Filter commits by a specific author.

```
bash
Copy code
git log --author="John Doe"
```

- `-since=<date>` / `-until=<date>`: Show commits within a date range.

```
git log --since="2 weeks ago"
```

- `p`: Show the patch/diff introduced by each commit.

```
git log -p
```

git show

The git show command displays detailed information about a specific commit, including the changes made in that commit.

`git show <commit-hash>`

Example:

git show **a1b2c3d4e5f6g7h8i9j0k**

commit a1b2c3d4e5f6g7h8i9j0k (HEAD -> main)
Author: Your Name
your.email@example.com
Date: Tue Jan 6 14:15:20 2025 +0000

Fixed bug in user authentication
diff --git a/app/user_auth.py b/app/user_auth.py
index 123abc4..567def8 100644
--- a/app/user_auth.py
+++ b/app/user_auth.py
@@ -20,7 +20,7 @@ def authenticate_user(username, password):
if not user:
return False

- if not verify_password(password, user.password_hash):
- if not verify_password_strength(password, user.password_hash):
  return False

  return True

return True

This shows the commit's metadata (author, date, and message) and the diff (differences) between the previous commit and the current commit.

Amending Commits (`git commit --amend`)

The `git commit --amend` command is used to modify the most recent commit. This can be useful when you want to:

- Modify the commit message.
- Add or remove changes from the commit.
- This is useful for correcting mistakes, updating commit messages, or adding changes.

`git commit --amend`

Use Case 1: Modify the Commit Message

If you want to change the commit message of the last commit:

1. Run:

git commit --amend

Git will open your default editor (e.g., Vim, Nano) with the existing commit message.

Modify the commit message as needed.

Save and close the editor.

Use Case 2: Add Changes to the Last Commit

You can amend the most recent commit to include additional changes:

1. Make the changes you need in your files.
2. Stage the changes:

git add <file-name>

1. Run:

git commit --amend

1. Git will open the commit message editor. The default message will be the same as the previous commit.

- You can either edit the message or leave it unchanged.

1. Save and close the editor. The changes are now part of the amended commit.

Use Case 3: Combining All Changes into a Single Commit

- Stage all changes:

  git add .

- Amend the commit:

```
git commit --amend
```

Use Case 4 : Remove Changes from the Last Commit

You can unstage files that were accidentally included in the last commit:

1. Unstage the files:
   ```
   git reset <file-name>
   ```

2. Amend the commit:
   ```
   git commit --amend
   ```

3. This will remove the changes from the commit but keep them in your working directory.

Important Notes:

History Rewriting:

- Amending rewrites history by replacing the original commit with a new one.
- Avoid using `-amend` for commits already pushed to a shared repository to prevent conflicts.
- Instead of amending, you can create a new commit if the changes need to be preserved separately, especially in shared repositories.

Force Push Required (if already pushed):

- If you've pushed the original commit, you'll need to force-push after amending:
  ```
  git push --force
  ```

Undoing Changes

1 Unstaging Files (`git reset`)

When you add a file to the staging area using `git add`, you can remove it from staging without losing changes in your working directory.

Command

`git reset <file>`

Example

```
git add file.txt
git reset file.txt
```

This removes
`file.txt` from the staging area but keeps its changes in your working directory.

2 Reverting Commits (`git revert`)

`git revert` creates a new commit that undoes the changes introduced by a previous commit. It's safe to use on shared branches as it doesn't alter the commit history.

Command

`git revert <commit-hash>`

Example

`git revert a1b2c3d`

This creates a new commit that undoes the changes made by commit `a1b2c3d`.

Key Points

- It doesn't remove the commit from history.
- Used for undoing changes in a collaborative environment.

✅ Revert with a Custom Commit Message

```
git revert -e <commit_hash>
```

The `-e` flag lets you **edit the commit message** before finalizing the revert.

✅ Revert Without Opening Editor

```
git revert --no-edit <commit_hash>
```

⚠️ Important Notes

Point	Explanation
`git revert` is safe	It's meant for public/shared branches.
Doesn't change commit history	It adds a **new commit** instead.
Can create merge conflicts	If the commit being reverted conflicts with later changes.

❌ When Not to Use `git revert`

Use `git reset` instead if:

- You are working on a **private branch**.
- You want to **completely remove commits** from history.
- You haven't pushed the commits yet.

3 Resetting Commits (`git reset`)

`git reset` moves the current branch pointer backward, effectively removing commits.

`git reset` is a Git command used to **undo local changes** by **resetting the HEAD** to a specified commit. Depending on how you use it, it can:

- Move the HEAD (i.e., current branch pointer)
- Modify the **staging area (index)**
- Alter or preserve the **working directory (your files)**

Reset Type	HEAD	Staging Area (Index)	Working Directory	Use Case

`--soft`	✅ Move	❌ Keep	❌ Keep	Rewriting commits but keeping changes staged
`--mixed`	✅ Move	✅ Reset	❌ Keep	Default; unstage changes but keep files
`--hard`	✅ Move	✅ Reset	✅ Discard	Completely undo commits and discard changes

It has three modes:

Soft Reset (--soft): Keeps changes in the staging area.

git reset --soft <commit-hash>

Mixed Reset (default): Keeps changes in the working directory but un stages them.

git reset <commit-hash>

Hard Reset (--hard): Discards all changes and resets the working directory to the specified commit.

git reset --hard <commit-hash>

🔹 1. `git reset --soft`

Moves `HEAD` to a previous commit but **keeps changes staged**.

✅ Use Case:

You made a commit too soon but want to edit it.

```
git reset --soft HEAD~1
```

🧩 This:

- Removes the last commit
- Keeps all changes in **staged (index)** state
- You can now fix and recommit

Example

To reset to a specific commit:

```
git reset --hard a1b2c3d
```

This moves the branch pointer to `a1b2c3d` and discards all changes after that commit.

🔹 2. `git reset --mixed` (default)

Moves `HEAD`, **unstages** the changes, but **keeps the working directory**.

✅ Use Case:

You want to modify your changes, but don't want them staged anymore.

```
git reset --mixed HEAD~1
```

🧩 This:

- Removes the last commit
- Keeps changes in your files
- Unstages them so you can edit before recommitting

🔹 3. `git reset --hard`

🚨 **Dangerous!** This removes all commits, **unstages**, and **deletes changes** from the working directory.

✅ Use Case:

You want to throw away local changes completely (e.g., rollback after failed CI/CD pipeline test).

```
git reset --hard HEAD~1
```

🧩 This:

- Deletes the last commit
- Discards the changes **forever** (unless recovered from reflog)

🔁

Real-Time DevOps Scenarios

✅ Scenario 1: Rollback a Broken Build (Before Pushing)

Your latest commit breaks the Jenkins pipeline, and you want to undo it before pushing:

```
git reset --soft HEAD~1
```

Now fix the bug and re-commit:

```
# make your changes
git commit -m "Fixed bug in build"
git push
```

✅ Scenario 2: Unstage Files Before Commit

You staged many files accidentally. Use:

```
git reset
```

This is equivalent to `--mixed`, and unstages the files.

✅ Scenario 3: Completely Wipe Local Changes

After a merge gone wrong or unstable experimental code, you want a **clean slate**:

```
git reset --hard origin/main
```

> This resets your local branch to exactly match origin/main.

✅ Scenario 4: Reset to a Specific Commit

To reset to a known good commit:

```
git log --oneline
# Find commit hash: a1b2c3d

git reset --hard a1b2c3d
```

What is
`git reflog`?

`git reflog` (short for **reference log**) is a powerful Git command that **tracks updates to the HEAD and branch references** — even **those that are not visible** in your normal commit history (`git log`).

> 🛠️ In simple terms:
>
> `git reflog` helps you **recover lost commits** or navigate through **your Git actions history** — including resets, reverts, rebases, and even commits that were deleted or orphaned.

📌 Why is `git reflog` important?

Use Case	Why It Helps
🔄 You did a `git reset --hard` and lost changes	Reflog shows the commit you were at before the reset — so you can go back.
🔁 You rebased and want to undo it	Shows the original HEAD before rebase.
❌ You deleted a branch accidentally	Reflog keeps a reference to it, even after deletion.
😨 You committed to the wrong branch	Reflog tracks it, so you can cherry-pick or move it.

🧪 Example Scenario: Recover a Commit After `reset --hard`

1. Suppose your commit history looks like:

```
A → B → C (HEAD)
```

2. You run:

```
git reset --hard B
```

3. Now commit C is **gone** from `git log`, but still accessible via `git reflog`:

```
git reflog
```

   Output:

```
6e8d3f0 HEAD@{0}: reset: moving to HEAD~1
a1b2c3d HEAD@{1}: commit: Add deployment script
...
```

4. You can restore it:

```
git reset --hard HEAD@{1}
```

🛠️ Common `git reflog` Commands

Command	Purpose
`git reflog`	Show all recent HEAD updates
`git reflog show <branch>`	Show reflog of a specific branch
`git branch restore-point HEAD@{3}`	Create a branch from a reflog reference

* Stashing Changes

In Git, when you're working on a feature or configuration and haven't committed your changes yet, sometimes you're forced to **pause** that work and **switch to another branch** or **task**.

🔧 The Problem:

You have **uncommitted changes** in your working directory, but Git won't let you switch branches without either committing or discarding those changes.

Example message:

```
error: Your local changes to the following files would be overwritten by check out
```

🔐 The Solution:

That's where `git stash` comes in.

🎯 Definition:

> git stash temporarily saves ("stashes") your uncommitted changes, restores your working directory to a clean state, and lets you come back to your work later.

Think of it like pressing "pause" on your current changes so you can safely switch tasks.

What is the Stash?

The Git stash is a temporary storage area where you can save your changes that are not ready to be committed. It allows you to work on something else without committing or discarding your current progress.

Key Features:

- Saves changes from both the working directory and the staging area.
- Temporarily removes the changes from your working directory so you can switch branches or work on other tasks.
- You can retrieve the stashed changes later and continue where you left off.

🔬 What Gets Saved in a Stash?

By default, `git stash` saves:

- ✅ Modified files (in the working directory)
- ✅ Staged files (in the index)

With flags, you can also save:

- ✅ Untracked files (with `u`)
- ✅ Ignored files (with `a`)

And all of this is saved as a **new stash entry** in a special internal Git list (called the reflog).

🔄 What Happens When You Stash?

When you run `git stash`, Git does three things:

1. Takes a snapshot of your **modified and staged** changes.
2. Stores it in a **stack-like structure** (like saving a state in RAM).
3. **Reverts your working directory** to the latest commit (clean state).

You can then:

- Switch branches
- Work on something else
- Later **re-apply** the changes with `git stash pop` or `git stash apply`

📦 Git Stash = Temporary Workspace Manager

`git stash` is not a commit, not a branch — it's a **temporary, stack-based, versioned workspace**.

Every stash you create is stored like:

stash@{0} → Most recent
stash@{1} → Older
stash@{2} → Even older
...

✅ `git stash -u`

Also include **untracked** files (like new YAML or `.env` files)

git stash -u

🔍 Use Case: Stash both modified files and newly added config files not yet committed.

✅ `git stash list`

View the list of all your saved stashes.

```
git stash list
```

🧾 Output:

```
stash@{0}: WIP on feature/auth: login module refactor
stash@{1}: On main: added GitHub actions
```

✅ `git stash show`

Shows what was changed in the most recent stash.

```
git stash show
```

Add `-p` for full diff:

```
git stash show -p
```

Stashing Changes (`git stash`)

To save your changes into the stash, use the `git stash` command.

Command:

```
git stash -u -m "WIP: Jenkins pipeline improvements"
```

Example:

Modify files

`echo "Some changes" >>` Jenkins pipeline improvements `.txt`

Stash the changes

git stash -u -m "WIP: Jenkins pipeline improvements”

This saves your changes and reverts the working directory to a clean state.

Options

- **Include untracked files**:

```
git stash -u
```

This stashes both tracked and untracked files.

- **Stash only staged changes**:

```
git stash --keep-index
```

Applying and Dropping Stashes (`git stash apply` , `git stash drop`)

✅ `git stash apply`

Re-applies the stashed changes **but keeps them in stash list**.

```
git stash apply stash@{1}
```

If you have multiple stashes, you can specify the stash by its name (e.g., `stash@{0}`):

```
git stash apply stash@{0}
```

✅ `git stash drop`

After applying a stash, it remains in the stash list unless you explicitly remove it:

```
git stash drop
```

To drop a specific stash:

```
git stash drop stash@{0}
```

Example

```
git stash apply
git stash drop
```

Alternatively, apply and drop a stash in one step:

```
git stash pop
```

Scenario:

You're working on a branch called `feature-branch` and have made some changes to `file1.txt` and `file2.txt`. You need to temporarily stash these changes to work on a quick bug fix in another branch.

Steps and Commands:

1. **Make changes to files:**

   ```
   echo "Work in progress for feature" >> file1.txt
   echo "Additional feature changes" >> file2.txt
   ```

2. **Stash the changes:**

```
git stash
```

Output:

```
Saved working directory and index state WIP on feature-branch: abcdef1 Initial commit message
```

3. **Verify the stash list:**

```
git stash list
```

Output:

```
stash@{0}: WIP on feature-branch: abcdef1 Initial commit message
```

4. **Switch to another branch:**

```
git checkout bugfix-branch
```

5. **Fix the bug and commit the changes:**

```
echo "Bug fix" >> bugfix.txt
git add bugfix.txt
```

```
git commit -m "Fix critical bug"
```

6. **Switch back to the original branch:**

```
git checkout feature-branch
```

7. **Apply the stashed changes:**

```
git stash apply
```

Output:

```
On branch feature-branch
Changes not staged for commit:
   modified: file1.txt
   modified: file2.txt
```

8. **Drop the applied stash:**

```
git stash drop
```

Output:

```
Dropped stash@{0} (WIP on feature-branch: abcdef1 Initial commit messa
```

```
ge)
```

Alternative: Apply and Drop in One Step

✅
`git stash pop`

Instead of applying and then dropping, you can use `git stash pop` to apply the stash and remove it simultaneously:

```
git stash pop
```

Output:

```
On branch feature-branch
Changes not staged for commit:
   modified: file1.txt
   modified: file2.txt
Dropped stash@{0} (WIP on feature-branch: abcdef1 Initial commit message)
```

Result:

Your previously stashed changes are now back in the working directory, and the stash has been removed.

4 Listing Stashes (`git stash list`)

To view all stashed entries:

```
git stash list
```

Example

```
git stash list
```

Output:

```
stash@{0}: WIP on main: 1234567 Add new feature
stash@{1}: WIP on feature: abcdefg Fix bug
```

- `stash@{0}`: The most recent stash.
- `stash@{1}`: The second most recent stash.

Advanced Usage

- **View the contents of a stash**:

  ```
  git stash show -p stash@{0}
  ```

✅
`git stash clear`

  ```
  git stash clear
  ```

✅ `git stash branch <new-branch-name>`

Creates a new branch and **applies a stash to it**.

```
git stash branch fix-docker stash@{1}
```

Step-by-Step Solution Using Stash:

```
bash
CopyEdit
# Step 1: Save your current work (including untracked YAML)
git stash -u -m "WIP: CI/CD optimization"

# Step 2: Verify stash list
git stash list
# stash@{0}: On feature/cicd: WIP: CI/CD optimization

# Step 3: Switch to main and do hotfix
git checkout main
vi Jenkinsfile
git add .
git commit -m "Fix: prod pipeline broken stage"
git push origin main

# Step 4: Switch back and apply changes
git checkout feature/cicd
git stash apply stash@{0}

# Step 5: If apply worked, remove it from stash
git stash drop stash@{0}
```

Viewing Changes and Logs

Viewing Changes (`git diff`)

The `git diff` command shows the differences between changes in your working directory, staging area, or between commits.

Key Use Cases

1. **View unstaged changes:**

```
git diff
```

Shows changes in the working directory that haven’t been staged yet.

2. **View staged changes:**

```
git diff --staged
```

Shows differences between the staging area and the last commit.

3. **Compare two commits:**

```
git diff <commit-hash1> <commit-hash2>
```

Shows differences between two specific commits.

4. **Compare a branch to the current branch:**

```
git diff <branch-name>
```

Shows changes in the given branch compared to the current branch.

Example

```
git diff HEAD
```

Shows differences between the working directory and the latest commit.

Comparing Commits

You can compare two commits to see what has changed between them.

Command

```
git diff <commit-hash1> <commit-hash2>
```

Example

```
git diff abc123 def456
```

This compares the changes between commit `abc123` and `def456`.

Additional Options

- **Show only file names that changed:**

  ```
  git diff --name-only <commit-hash1> <commit-hash2>
  ```

- **Show changes for a specific file:**

  ```
  git diff <commit-hash1> <commit-hash2> -- <file-name>
  ```

Using `git blame`

The `git blame` command shows line-by-line annotations of a file, identifying the commit and author responsible for each line.

Command

```
git blame <file-name>
```

Example

```
git blame file.txt
```

Output:

```
a1b2c3d (Author Name 2024-07-01) Line 1 content
e4f5g6h (Another Author 2024-07-02) Line 2 content
```

Options

1. **Show changes only since a specific commit:**

   ```
   git blame <file-name> <commit-hash>
   ```

2. **Ignore whitespace changes:**

   ```
   git blame -w <file-name>
   ```

3. **Blame for specific lines in a file:**

```
git blame -L <start>,<end> <file-name>
```

Example:

```
git blame -L 10,20 file.txt
```

Shows annotations only for lines 10 to 20.

Practical Use Case

📌 git diff

```
bash
CopyEdit
git diff              # Unstaged changes
git diff --staged     # Staged but not committed
git diff HEAD         # Working dir vs latest commit
git diff commit1 commit2
git diff branch1..branch2
git diff commit1 commit2 -- file.txt
git diff --name-only commit1 commit2
```

📌 git blame

```
bash
CopyEdit
git blame file.txt                  # Full blame
git blame -L 10,20 file.txt         # Blame lines 10–20 only
```

```
git blame -w file.txt                # Ignore whitespace
git blame file.txt abc1234           # Show blame up to specific commit
```

Tracked Files Vs Untracked Files

🔍 Why This Matters in Real Projects

In any Git-based project — whether you're a **DevOps engineer managing infra scripts** or a **developer building features** — understanding **how Git categorizes files** helps you:

- Prevent **accidental commits** (like `.env`, credentials, test logs)
- Manage **build artifacts**, secrets, and temp files
- Keep CI/CD pipelines clean
- Avoid errors like “file not added” or “unexpected file removed”

In Git, files in a repository can be categorized into **tracked** and **untracked** files.

1. Tracked Files

These are files that Git is aware of and manages. Tracked files can be in one of the following states:

- **Unmodified**: Files that haven’t changed since the last commit.
- **Modified**: Files that have been changed but not yet staged.
- **Staged**: Files that are marked to be included in the next commit (added to the staging area).

Example Commands:

- `git add <file>` → Moves modified files to the **staged** state.
- `git commit -m "message"` → Saves staged changes to the repository.

2. Untracked Files

These are files that Git does **not** track yet. They are not part of any previous commits and are not staged for the next commit.

Example Commands:

- `git status` → Lists untracked files.
- `git add <file>` → Moves untracked files to the **staged** state.
- `git clean -f` → Removes untracked files from the working directory.

🔄 Git Lifecycle for Files

```
UNTRACKED --(git add)⟶ STAGED --(git commit)⟶ TRACKED (Unmodified)
   ↑                          ↓
   |                          |
   +-------------(deleted or clean)----------+
```

🔐 Real-Time Examples

🛠️ DevOps Engineer Use Case

Scenario:

You're managing Kubernetes YAMLs, and you create a new `autoscaler.yaml`.

```
$ ls
deployment.yaml  service.yaml  autoscaler.yaml
```

You run:

```
git status
```

📍 Output:

```
ntracked files:
  autoscaler.yaml
```

You forget to `git add autoscaler.yaml`, push changes, and the deployment fails because the file wasn't tracked or committed.

🛠️ **Fix**:

```
git add autoscaler.yaml
git commit -m "Added HPA autoscaler config"
```

💻 Developer Use Case

Scenario:

You write a new test file `login.test.js`.

But CI pipeline fails because:

- Test file wasn't committed (was untracked)
- Your `.gitignore` accidentally included `.test.js`

You run:

```
git status
```

📍 You realize it was never staged:

```
Untracked files:
  login.test.js
```

🛠️ Fix:

```
git add login.test.js
git commit -m "Added login test case"
```

📦 Useful Commands: Full Cheat Sheet

Command	Purpose
`git status`	Shows tracked/untracked and staging status
`git add <file>`	Move untracked/modified file to staging
`git commit -m "msg"`	Commit all staged changes
`git rm <file>`	Remove tracked file
`git clean -f`	Delete **untracked** files (DANGEROUS)
`git clean -fd`	Delete untracked files **and directories**
`git ls-files`	Lists all tracked files
`git diff`	Shows what's changed (but not staged)
`git diff --cached` or `--staged`	Shows changes in staging area

Branching and Merging